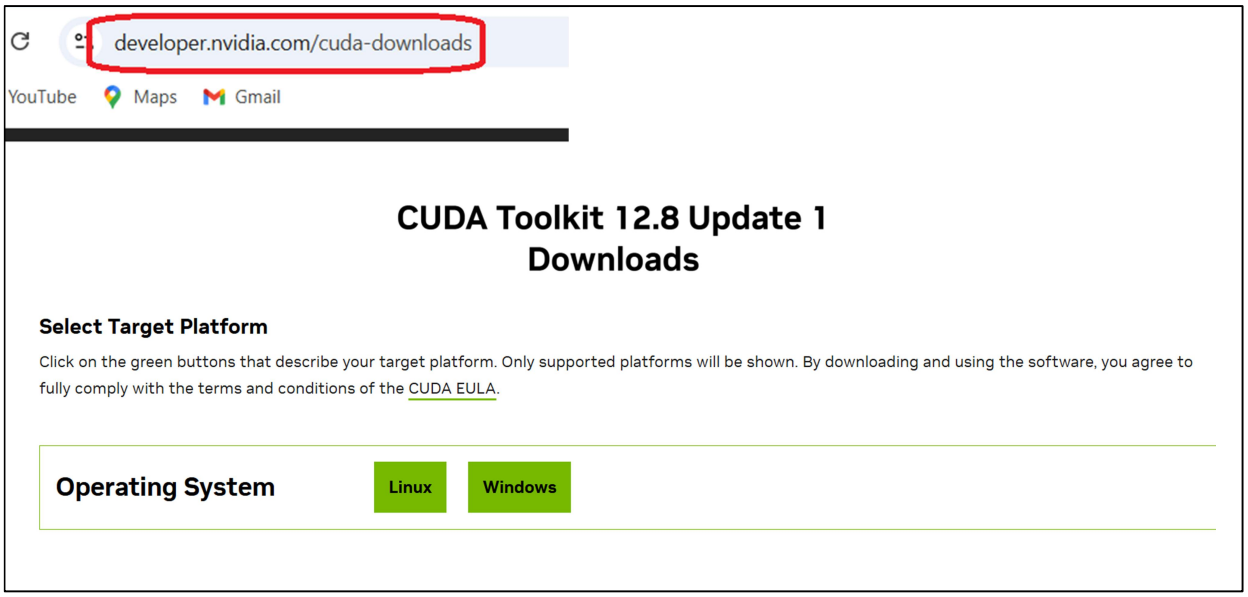
Installation de CUDA et Test d'une application CUDA avec Visual Studio

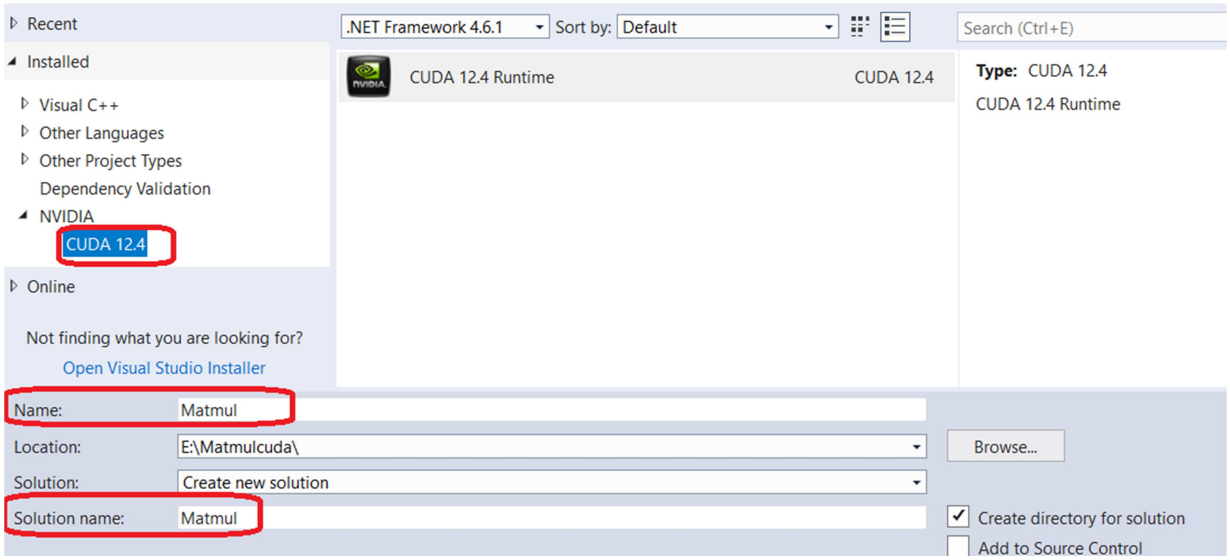
(Matrix Multiplication) – à préparer avant la séance

1- Il faut déjà avoir Visual Studio ver 17 ou plus.

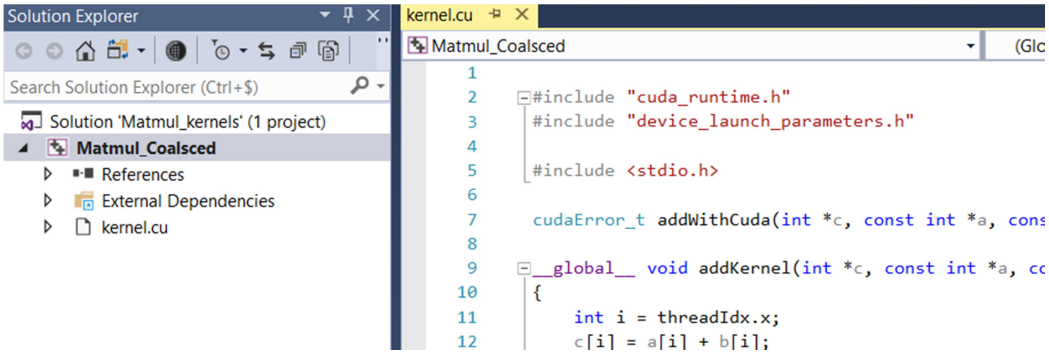
2- Télécharger et installer NVIDIA Toolkit . Redémarrer le PC.



3- Créer un nouveau projet Visual Studio « ***NVIDIA CUDA*** ».
Choisir un nom pour la solution, ainsi que pour le projet



4- Le nouveau projet créé contient un fichier ayant l'extension .**cu** (CUDA). Ce fichier contient un code par défaut avec un kernel permettant d'effectuer une opération d'addition.



5- Remplacer le code existant par le code contenu dans le fichier « **kernel.cu** » , joint avec ce document pdf.

6- Au niveau du fichier .cu, se trouvent

- deux kernels (**matrixMulXrow** et **matrixMulYrow**).

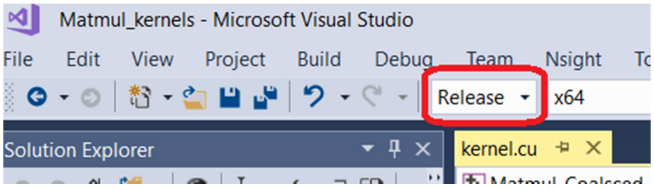
```
__global__ void matrixMulXrow(const float* a, const float* b, float* c, int N) {
    // Compute each thread's global row and column index
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    int col = blockIdx.y * blockDim.y + threadIdx.y;

/* ================================================================================ */
__global__ void matrixMulYrow (const float* a, const float* b, float* c, int N) {
    // Compute each thread's global row and column index
    int row = blockIdx.y * blockDim.y + threadIdx.y;
```

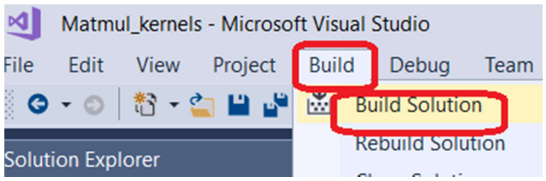
- une fonction **main**, ou se fait l'invocation de l'un des Kernels (**matrixMulXrow**)

```
// Launch kernel
matrixMulXrow << <blocks, threads >> > (d_a, d_b, d_c, N);
```

7- Sélectionner le mode « Release »



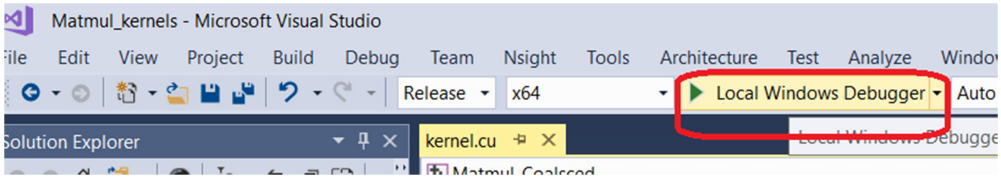
8- Compiler le programme (Build Solution)



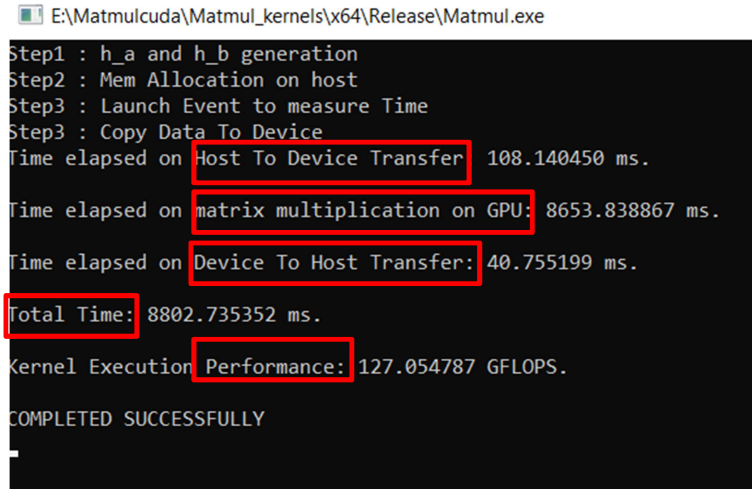
Normalement, la compilation devrait se passer sans aucun problème

```
1>Matmul_Coalsced.vcxproj -> E:\Matmulcuda\Matmul_kernels\x64\Debug\Matmul_Coalsced.exe
1>Done building project "Matmul_Coalsced.vcxproj".
========== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ==========
```

9- Lancer l'exécution en appuyant sur « **Local Windows Debugger** »



Une fois lancé, le temps d'exécution, des transferts du Host au Device et inversement, ainsi que la performance en GFLPOS seront affichés.
Appuyer sur une touche pour terminer l'exécution.



10- Au niveau de la fonction main, changer la commande d'invocation (kernel **matrixMulYrow** au lieu de matrixMulXrow) . L'exécuter.

```
// Launch kernel
matrixMulYrow << <blocks, threads >> > (d_a, d_b, d_c, N);
```